MACHINE LEARNING

(CSDC-0334)

Lab File

B. Tech – CSE

6th Semester

Section-B

Department of Computer Science and Engineering
Dr. B. R. Ambedkar National Institute of Technology, Jalandhar,
Punjab, 144008

(January-June 2026)

Submitted to
Dr. Sourav Jindal

Submitted by:
Name : Tahira Mehta
Roll No : 23103146
Group : G7

Lab Assignment 1

Machine Learning Lab - CSDC0336

(NITJ-CSED)

Learning Goals:

- Apply basic Python programming skills using loops and lists.
- Understand the levels of intelligence in data science and their real-world applications.
- Perform basic data preprocessing including handling missing values and encoding categorical data.

Task 1: Given a list arr = [2, 0, 1, 2, 0, 1], write a Python script using a while loop to sort the numbers so all **0s** come first, then **1s**, and finally **2s**

Task 2: In Machine Learning, we sometimes need to generate thousands of random data points to test our models. Write a script that uses a for loop to generate and print 10 random numbers between 1 and 100.

Task 3: Choose a unique industry and explain how it uses the four levels of intelligence: Statistics, AI, Machine Learning, and Deep Learning.

Task 4: Load the given dataset and perform basic data preprocessing steps including handling missing (NA) values, imputing numeric and categorical data, and encoding categorical variables using appropriate techniques. The goal is to prepare a clean, machine-learning-ready dataset and understand the importance of preprocessing in ML pipelines.

Dataset Link: - https://github.com/datasciencedojo/datasets/blob/master/titanic.csv

TASK 1 :

```
#counting the number of zeroes,ones,two's and appending them to the array
arr = [2, 0, 1, 2, 0, 1]
count0 = 0
count1 = 0
count2 = 0
i = 0
while i < len(arr):
    if arr[i] == 0:
        count0 += 1
    elif arr[i] == 1:
        count1 += 1
    else:
        count2 += 1
    i += 1
sorted_arr = []
```

```
i = 0
while i < count0:
    sorted_arr.append(0)
    i += 1
i = 0
while i < count1:
    sorted_arr.append(1)
    i += 1
i = 0
while i < count2:
    sorted_arr.append(2)
    i += 1
print("Sorted array:", sorted_arr)
```

```
•••  [0, 0, 1, 1, 2, 2]
```

TASK 2 :

```
import random
print("random numbers to be printed between 1 and 100")
for i in range (10):
  print(random.randint(1,100))
```

```
random numbers to be printed between 1 and 100
30
33
93
5
92
54
16
42
32
34
```

TASK 3 :

Industry Chosen: Smart Agriculture (Precision Farming)

Smart Agriculture uses modern data-driven technologies to increase crop yield, reduce waste, and improve decision-making. It applies all four levels of intelligence in a layered manner:

---

1. Statistics

Statistics is the base of smart farming.

- Farmers collect data like rainfall, soil moisture, temperature, humidity, and crop yield.
- Using statistics, they find:
    - Averages
    - Patterns
    - Relationships between data

- Example:
    - Calculating how much rainfall is needed on average to grow wheat

---

2. Artificial Intelligence (AI)
AI works using fixed rules given by humans.
- It uses expert systems that follow predefined rules.
- These rules help farmers decide what action to take.
- Example rule:
    - IF soil moisture is low AND temperature is high → start irrigation
- AI systems act like human agriculture experts.

Purpose:
To make decisions automatically using human-made rules.

---

3. Machine Learning (ML)
Machine Learning learns from data instead of fixed rules.
- ML models are trained using old farming data.
- They can:
    - Predict crop yield
    - Identify plant diseases
    - Suggest the best time for irrigation
- Example:
    - The system learns which soil, weather, and fertilizer give the best crop yield

Purpose:
To improve results automatically as more data is collected.

---

4. Deep Learning (DL)
Deep Learning is used for complex data like images and videos.
- It uses neural networks and data from drones or satellites.
- It helps in:
    - Detecting crop diseases from leaf images
    - Finding pest attacks using cameras
- Example:
    - A deep learning model finds leaf disease early by looking at images

Purpose:
To analyze large and complex data with high accuracy.

<u>TASK 4 :</u>

```
import pandas as pd
import numpy as np
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
print("Initial Data Info:")
print(df.info())
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

```
df.drop(columns=['Cabin'], inplace=True)
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
print("\nCleaned Dataset Info:")
print(df.info())
print("\nSample data:")
print(df.head())
```

```
Initial Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None

Cleaned Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
...
1          PC 17599  71.2833       False       False
2  STON/O2. 3101282   7.9250       False        True
3            113803  53.1000       False        True
4            373450   8.0500       False        True
```